ESPECIFICACIÓN DE ARQUITECTURA Y CONTRATO DE INTERFAZ

API de Gestión de Contenido del DevBlog

devblog-cms-api · API-First en 6 pasos + implementación y servicios

Autor	Leonardo Nucamendi Ruiz	**Versión**	1.2 (definitiva)
Proyecto	CMS del blog personal DevBlog_	**Fecha**	Octubre 2026

Cómo leer este documento

Este documento especifica, antes de escribir código, el contrato de la API que administrará los artículos de mi blog DevBlog_. Sigue la metodología API-First en 6 pasos secuenciales: cada paso es el insumo del siguiente, desde el contexto del servicio hasta el detalle de cada endpoint. Al final de cada paso se incluye una caja de **Justificación de diseño** que explica el porqué de las decisiones, un Anexo A muestra cómo evolucionó mi boceto original hacia este contrato y un Anexo B resume los cambios de la v1.0 a la v1.1 y un Anexo C los de la v1.1 a la v1.2. Un Anexo D documenta la implementación de referencia, la conexión con el blog, los servicios del ecosistema y las decisiones operativas de la entrega.

Simbología de modificadores

`!` Obligatorio	`?` Opcional	`#` Solo lectura (generado por el sistema)	`@` Inmutable tras crearse

Convenciones. Los identificadores técnicos (campos, rutas, enumeraciones) se escriben en inglés y `snake_case`, igual que en el ejemplo de referencia; el texto explicativo está en español. Los estados se escriben en mayúsculas (`BORRADOR`, `PUBLICADO`). Todas las fechas usan ISO 8601 en UTC.

PASO 1 DE 6: Contexto y Objetivo General

Servicio	API de Gestión de Contenido del DevBlog (`devblog-cms-api`).
Objetivo	Administrar el ciclo de vida de los artículos del blog personal DevBlog_, desde su creación como borrador hasta su publicación, y exponer métricas cuantitativas de lectura (vistas y tiempo de lectura) de los artículos ya publicados.
Límites (No hace)	• No renderiza ni sirve las páginas HTML del blog; eso lo hace el Sitio Web Público. • No gestiona anuncios ni monetización: el DevBlog no inserta publicidad en sus páginas. • No gestiona usuarios, lectores, comentarios ni autenticación de visitantes (hay un solo autor). • No almacena archivos multimedia: el contenido solo puede referenciar URLs externas. • No calcula analítica avanzada (visitantes únicos, tiempo real de permanencia, rebote, procedencia).
Consumidores	• Consola de Administración del CMS (productor de contenido y consulta de métricas). • Sitio Web Público DevBlog_ (lectura de publicaciones y registro de vistas).
Nivel de exposición	Infraestructura interna: solo la consumen dos clientes propios y no se comercializa a terceros. Aun así, toda operación de escritura exige autenticación, porque el contenido del blog es la identidad profesional del autor.

Aplicación de los filtros de alcance

Consumidor	Qué necesita resolver	Operaciones que usa
Consola de Administración	Escribir, corregir y publicar artículos; retirar contenido; revisar cuántas lecturas tiene cada publicación.	Todas las operaciones de Borradores; `GET`/`DELETE` de Publicaciones; métricas.
Sitio Web Público	Mostrar la lista de artículos, abrir un artículo por su URL y contar cada lectura.	`GET /posts`, `GET /posts/slug/{slug}`, `POST /posts/{post_id}/views`.

> **Justificación de diseño**
>
> **Filtro del consumidor:** mi boceto incluía `POST /Posts/ads` para configurar anuncios por publicación, pero mi propia nota indica que no aplica al blog dev. Al no existir un consumidor que necesite esa capacidad, se descartó del contrato (y el atributo `Ads_Activos` salió del modelo). Esta exclusión se mantiene en la v1.1: ningún atributo ni endpoint de anuncios forma parte del contrato.
>
> **Filtro del nombre:** "Gestión de Contenido" abarca crear, editar, publicar y retirar artículos. Las métricas se mantienen porque son lectura del propio contenido gestionado; la analítica avanzada, los comentarios y los anuncios no son lógicos dentro de ese nombre y se declaran como exclusiones explícitas.

Estilo arquitectónico: REST

La API sigue el estilo REST sobre HTTP, con representaciones JSON (`application/json`). Se evaluó GraphQL como alternativa y se descartó por las razones siguientes.

Criterio	Aplicación al DevBlog	Conclusión
Dirección del tráfico	Los dos consumidores (Consola y Sitio Web Público) llaman al servicio desde fuera: tráfico vertical (norte-sur), el caso en que REST es adecuado.	REST
Madurez (Richardson)	URIs separadas por recurso (`/drafts`, `/posts`) y varios métodos HTTP sobre cada una: Nivel 2. No se llega al Nivel 3 (HATEOAS): con solo dos clientes propios que ya conocen el contrato, los controles hipermedia añadirían complejidad sin reducir la dependencia de la documentación.	Nivel 2

Escenario ideal para GraphQL	¿Aplica al DevBlog?
Móviles, relojes o IoT que cuidan cada kilobyte y la batería	No: los clientes son una consola web y un sitio web.
Fusionar bases de datos, sistemas heredados y APIs de terceros en una sola ventanilla	No: hay un solo almacén de datos y ninguna integración externa.
Datos profundamente conectados que en REST exigirían encadenar llamadas	No: el artículo es un solo recurso con atributos planos (las etiquetas incluidas); además, la lectura por slug ya evita la segunda llamada.
Prototipado rápido donde el cliente cambia la consulta sin tocar el servidor	No: el contrato se diseña primero (API-First) y es estable.

Los problemas que GraphQL resuelve ya están resueltos dentro de REST: el over-fetching se evita porque los listados omiten `content`, y el under-fetching se evitó en la v1.1 con `GET /posts/slug/{slug}`. Además, GraphQL suele responder `200` aunque la operación falle y complica el caché HTTP y el límite de tráfico por ruta, y este contrato depende de `404`/`409`/`422`, `Retry-After` y `ETag`.

Compromisos de REST que se asumen

Limitación (teoría)	Cómo se maneja en este contrato
Vocabulario limitado	Pasar de `BORRADOR` a `PUBLICADO` no tiene un verbo HTTP propio, así que se modela como `POST /drafts/{post_id}/publish` (una acción sobre el recurso). Se descartó `PATCH` con `status`, porque dejaría al cliente intentar transiciones arbitrarias; un endpoint dedicado concentra las invariantes 1, 4 y 6.
Dos colecciones del mismo recurso	`/drafts` y `/posts` exponen el mismo Post según su estado (los dos módulos del boceto original). Se mantienen porque separan a la Consola (borradores) del Sitio Web Público (publicaciones), con permisos y reglas distintos. Contrapartida: un Post publicado responde `409` en `/drafts`.
Ausencia de estado	Cada petición lleva su token Bearer y el servidor no guarda sesiones. Excepción: el limitador de tráfico y el registro de `Idempotency-Key` guardan estado por proceso; con varias instancias deben pasar a un almacén compartido (por ejemplo, Redis).

Limitación (teoría)	Cómo se maneja en este contrato
Sin versionado estandarizado	Prefijo `/api/v1`. Un cambio que rompa el contrato después de publicar la API (por ejemplo, cambiar el tipo de `post_id`) exigiría `/api/v2`; los cambios de la v1.0 a la v1.1 son previos a la publicación, por eso el prefijo no cambió.

PASO 2 DE 6: Glosario de Sustantivos del Dominio

Cada término se define con su significado exacto dentro de esta API, para evitar ambigüedades en el modelado y en los endpoints.

Entidad / Término	Definición en este contexto	ID
Post	Artículo del blog, almacenado como un único registro que transita por los estados `BORRADOR` y `PUBLICADO`. Un "borrador" y una "publicación" no son entidades distintas: son el mismo Post en distinto estado.	`post_id`
Módulo Drafts (Borradores)	Colección de rutas `/drafts` que opera únicamente sobre Posts en estado `BORRADOR`.	—
Módulo Posts (Publicaciones)	Colección de rutas `/posts` que opera únicamente sobre Posts en estado `PUBLICADO`.	—
Slug	Texto único, en minúsculas y con guiones, que identifica al Post dentro de la URL pública (ejemplo: `/blog/{slug}`) y permite leer la publicación directamente.	`slug`
Tag (Etiqueta)	Palabra clave en minúsculas que clasifica un Post (ejemplo: `tutorial`, `deployment`). No es una entidad independiente: es un atributo multivalor del Post.	`tags`
PostMetrics	Vista de solo lectura derivada de un Post `PUBLICADO`: contador de vistas y tiempo de lectura estimado. No tiene identidad propia.	`post_id`
View (Vista)	Evento unitario que representa una carga del artículo en el Sitio Web Público; cada evento incrementa `views_total` en 1.	— (evento)

> **Justificación de diseño**
>
> Separar "Drafts" y "Posts" como módulos, y no como entidades, respeta la decisión de mi boceto de centralizar todo en una sola tabla: el módulo es solo una forma de exponer el mismo registro según su estado.
>
> `Tag` no se modela como entidad porque no tiene atributos propios ni ciclo de vida; crear una tabla adicional sería agregar complejidad sin justificación.

PASO 3 DE 6: Modelado de Datos e Invariantes

Entidad: Post (Estados: BORRADOR → PUBLICADO)

Persistencia: una sola tabla (`Publicacion_Blog`) donde `status` actúa como discriminador entre borradores y publicaciones. El ciclo es de una sola vía: un Post publicado no regresa a borrador.

Atributo	Mod.	Tipo / Estructura	Nota
`post_id`	`!#@`	String UUIDv7 (36 caracteres)	Identificador único no secuencial, generado por el sistema.
`title`	`!`	String, 5–120 caracteres	Editable solo en `BORRADOR`.
`slug`	`!`	String kebab-case, 3–100 caracteres	Único global. Editable solo en `BORRADOR`; pasa a inmutable al publicar.
`content`	`!`	String (Markdown), 1–100,000 caracteres	Cuerpo del artículo. Mínimo 100 caracteres para publicar.
`tags`	`?`	List[0..5] de String	Sin duplicados. Editable solo en `BORRADOR`.
`status`	`!#`	Enum [`BORRADOR`, `PUBLICADO`]	Solo cambia mediante `POST /drafts/{post_id}/publish`.
`created_at`	`!#@`	ISO 8601 UTC	Fecha de creación del borrador.
`updated_at`	`!#`	ISO 8601 UTC	Se actualiza en cada modificación mientras es `BORRADOR`.
`published_at`	`?#@`	ISO 8601 UTC	Nulo mientras es `BORRADOR`; se fija una sola vez al publicar.
`views_total`	`!#`	Integer ≥ 0	Inicia en 0; solo crece con eventos View.
`reading_time_min`	`#`	Integer ≥ 1 (derivado)	Calculado al leer: ⌈palabras de `content` / 200⌉. No se persiste.

Transiciones válidas del ciclo de vida

Estado origen	Acción (endpoint)	Estado destino	¿Permitida?
(nuevo)	`POST /drafts`	`BORRADOR`	Sí
`BORRADOR`	`PUT /drafts/{post_id}` (iterar)	`BORRADOR`	Sí
`BORRADOR`	`POST /drafts/{post_id}/publish`	`PUBLICADO`	Sí
`BORRADOR`	`DELETE /drafts/{post_id}`	(eliminado)	Sí
`PUBLICADO`	`GET` publicación / métricas / `POST` vista	`PUBLICADO`	Sí
`PUBLICADO`	`DELETE /posts/{post_id}`	(eliminado)	Sí, con confirmación
`PUBLICADO`	Cualquier operación de `/drafts`	`BORRADOR`	No (409)

Restricciones e invariantes de la entidad

#	Invariante	Descripción
1	Ciclo unidireccional	Un Post en `PUBLICADO` jamás puede volver a `BORRADOR`. No existe la operación de "despublicar".
2	Inmutabilidad editorial	En `PUBLICADO`, `title`, `slug`, `content` y `tags` no pueden modificarse; el módulo Posts no expone ninguna operación de edición.
3	Unicidad de slug	No pueden coexistir dos Posts con el mismo `slug`, sin importar su estado. Así un borrador no puede "robar" la URL de una publicación y la lectura por slug devuelve a lo sumo un Post.
4	Completitud para publicar	Un Post solo transita a `PUBLICADO` si su `title` y `slug` son válidos y `content` tiene al menos 100 caracteres.
5	Vistas monótonas	`views_total` inicia en 0, solo aumenta de 1 en 1 por cada evento View y nunca decrece. Solo un Post `PUBLICADO` acumula vistas.
6	Marcas de tiempo del sistema	`created_at` nunca cambia; `updated_at` se renueva con cada edición del borrador; `published_at` se fija una sola vez y siempre es ≥ `created_at`.
7	Visibilidad	Un Post en `BORRADOR` nunca es visible para el Sitio Web Público: los endpoints de lectura pública responden como si no existiera (404).
8	Integridad de etiquetas	Máximo 5 etiquetas por Post, sin duplicados; cada una en minúsculas, de 2 a 30 caracteres, solo letras, dígitos y guiones.
9	Identificador opaco	`post_id` es un UUIDv7 generado por el servidor, nunca secuencial, y no puede asignarlo ni modificarlo el cliente.

Justificación de diseño

Una sola tabla: el volumen es bajo (un autor) y las consultas siempre filtran por `status`, por lo que separar borradores y publicaciones en tablas distintas solo añadiría uniones y duplicación. El atributo clave del modelo sigue siendo el estado, como en mi boceto.

`post_id` como UUIDv7 (cambio de la v1.1): un entero autoincremental expone el volumen de la base de datos: cualquier visitante podría recorrer `/posts/1`, `/posts/2`, `/posts/3`... y deducir cuántos artículos existen (publicados y borradores) y a qué ritmo se escribe. Con un UUIDv7 los identificadores no se pueden adivinar ni enumerar, por lo que el total de artículos deja de ser inferible desde fuera. Además, al ser ordenable en el tiempo conserva buen rendimiento en índices y sirve como desempate estable en la paginación. Contrapartida: un UUIDv7 incluye una marca de tiempo aproximada de creación; se acepta porque revela cuándo se creó un artículo, no cuántos hay.

Inmutabilidad al publicar: el `slug` forma parte de la URL pública; cambiarlo rompería enlaces ya compartidos. Se acepta como contrapartida que corregir una errata en una publicación exige eliminarla y recrearla.

Nuevos atributos: `updated_at` y `published_at` no estaban en el boceto; son necesarios para ordenar listados y para calcular métricas de antigüedad.

PASO 4 DE 6: Diccionario de Tipos de Datos

Catálogo centralizado: cada dato tiene un solo nombre y un solo formato en toda la API.

Campo	Tipo	Formato / Regla	Ejemplo
`post_id`	string	UUIDv7 estándar de 36 caracteres, generado por el servidor.	`0192a3b5-1122-7334-8445-556677889900`
`title`	string	Texto sin saltos de línea, 5–120 caracteres (se recortan espacios en los extremos).	`Despliega tu web gratis en Vercel`
`slug`	string	Regex `^[a-z0-9]+(?:-[a-z0-9]+)*$`, 3–100 caracteres.	`despliega-tu-web-gratis-en-vercel`
`content`	string	Markdown en UTF-8, 1–100,000 caracteres (≥ 100 para publicar).	`## Introducción ...`
`tags`	array[string]	0–5 elementos únicos; cada uno con regex `^[a-z0-9-]{2,30}$`.	`["tutorial", "deployment"]`
`status`	string	Enum cerrado: `BORRADOR`, `PUBLICADO`.	`BORRADOR`
`created_at`	string	ISO 8601, fecha y hora UTC.	`2026-10-15T18:30:00Z`
`updated_at`	string	ISO 8601, fecha y hora UTC.	`2026-10-16T09:12:40Z`
`published_at`	string	ISO 8601, fecha y hora UTC. Ausente en borradores.	`2026-10-17T12:00:00Z`
`views_total`	integer	Entero mayor o igual a cero.	`128`

Campo	Tipo	Formato / Regla	Ejemplo
`reading_time_min`	integer	Entero ≥ 1; ⌈palabras / 200⌉.	`5`
`url`	string	Ruta relativa `/blog/{slug}`.	`/blog/despliega-tu-web-gratis-en-vercel`
`limit`	integer	Entre 1 y 50; por defecto 20.	`20`
`cursor` / `next_cursor`	string	Token opaco en base64url de un JSON con el `id` y el timestamp de ordenamiento del último elemento entregado (`updated_at` en borradores, `published_at` en publicaciones). `next_cursor` se omite en la última página.	`eyJpZCI6IjAxOTJhM2I1LTExMjItNzMzNC04NDQ1LTU1NjY3Nzg4OTkwMCIsInB1Ym`
`tag` (filtro)	string	Mismo formato que un elemento de `tags`.	`tutorial`
`confirm`	boolean	Debe ser `true` para eliminar una publicación.	`true`
`Idempotency-Key` (cabecera)	string	16–64 caracteres ASCII; se recomienda UUID v4.	`7c9e6679-7425-40de-944b-e07fc1f90ae7`
`error`	object	`{ code (! UPPER_SNAKE), message (! String), field (? String) }`	`{"code": "SLUG_DUPLICATED", "message": "El slug ya está en uso.",`

Estructura del cursor

El cliente trata el cursor como una cadena opaca; el servidor lo decodifica. El ejemplo de la tabla corresponde a un listado de publicaciones (endpoint 7) y decodifica a:

```
{"id": "0192a3b5-1122-7334-8445-556677889900", "published_at": "2026-10-15T18:30:00Z"}
```

En el listado de borradores (endpoint 2) el segundo campo es `updated_at`:

```
{"id": "0192a3b5-1122-7334-8445-556677889900", "updated_at": "2026-10-16T09:12:40Z"}
```

Ambos listados ordenan por el timestamp descendente y usan `post_id` como desempate; el cursor guarda el par (timestamp, id) del último elemento y la siguiente página devuelve los elementos estrictamente posteriores en ese orden. Un cursor mal formado, alterado o de otro listado responde `422`.

> **Justificación de diseño**
>
> **Montos, fechas y enums:** se usan formatos estándar (ISO 8601, enums cerrados) para que el Sitio Web Público y la Consola interpreten los datos sin ambigüedad.
>
> **Un solo nombre por dato:** el mismo concepto se llama `post_id` en cualquier ruta o respuesta, y no `id`, `draftId` o `postId` según el endpoint (como ocurría en mi boceto, donde la petición usaba `draftId`).
>
> **Cursor con id y timestamp (cambio de la v1.1):** como los listados se ordenan por fecha y no por id, un cursor que solo guardara el id no permitiría saber en qué posición del orden reanudar. Guardar solo la fecha tampoco basta, porque dos Posts pueden compartir el mismo timestamp y se perderían o repetirían elementos. Codificar el par (timestamp, id) da un orden total estable. Se mantiene opaco para poder cambiar su contenido sin romper a los clientes.
>
> **Formato de error único:** todas las respuestas de error usan el objeto `error`, lo que facilita mostrar mensajes de validación en la Consola.

PASO 5 DE 6: Especificación de Endpoints

Parámetros operativos pensados para una infraestructura de capa gratuita con tolerancia a arranque en frío. La API está organizada en dos módulos, igual que en el boceto: **Borradores** (`/drafts`) y **Publicaciones** (`/posts`).

Reglas transversales (aplican a todos los endpoints)

Base y formato	Prefijo `/api/v1`. Peticiones y respuestas en JSON UTF-8 (`Content-Type: application/json`).
Autenticación	• Cabecera `Authorization: Bearer <token>` en todas las peticiones. • Scopes: `cms:write` (Consola), `posts:read` (Sitio Web Público, lectura) y `posts:view` (Sitio Web Público, registro de vistas).
Límites de tráfico	Consola: 60 peticiones por minuto. Sitio Web Público: 10 peticiones por segundo. Al excederlos responde `429` con cabecera `Retry-After`.
Tiempo máximo	10 segundos por operación; si se excede responde `504`.
Idempotencia	Las operaciones `POST` que crean o cambian estado exigen `Idempotency-Key`. Misma clave dentro de la ventana indicada: devuelve el resultado original sin repetir la operación. Misma clave con un cuerpo distinto: `422`.
Caché HTTP	• Lecturas públicas (endpoints 7, 8 y 9): `Cache-Control: public, max-age=60` y `ETag` (hash del cuerpo). Si el cliente envía `If-None-Match` con el `ETag` vigente, responde `304 Not Modified` sin cuerpo. • Todo lo demás (borradores, métricas, vistas y errores): `Cache-Control: no-store`. • Contrapartida aceptada: una publicación eliminada puede seguir sirviéndose desde caché hasta 60 s.
Versionado	Prefijo `/api/v1`. Un cambio que rompa el contrato después de publicar la API exige un nuevo prefijo (`/api/v2`).
Paginación	Por cursor opaco (ver Paso 4). Orden descendente por `updated_at` (borradores) o `published_at` (publicaciones), con `post_id` como desempate.
Errores comunes	• `401` token ausente o inválido; `403` scope insuficiente. • `404` recurso inexistente o no visible para ese actor; `409` conflicto de estado o de unicidad; `422` validación de datos (incluye `post_id` que no sea un UUIDv7 válido).

Propiedades HTTP de cada endpoint

Un método es seguro si no modifica el estado del servidor, idempotente si repetirlo produce el mismo efecto y cacheable si su respuesta puede reutilizarse.

Endpoint	Método	Seguro	Idempotente	Cacheable
1. Crear borrador	POST	No	Sí, con `Idempotency-Key`	No
2. Listar borradores	GET	Sí	Sí	No
3. Obtener borrador	GET	Sí	Sí	No
4. Actualizar borrador	PUT	No	Sí	No

Endpoint	Método	Seguro	Idempotente	Cacheable
5. Eliminar borrador	DELETE	No	Sí (1.ª vez `204`; después `404`, mismo estado final)	No
6. Publicar borrador	POST	No	Sí, con `Idempotency-Key`	No
7. Listar publicaciones	GET	Sí	Sí	Sí
8. Obtener publicación por id	GET	Sí	Sí	Sí
9. Obtener publicación por slug	GET	Sí	Sí	Sí
10. Eliminar publicación	DELETE	No	Sí (misma regla que el 5)	No
11. Métricas	GET	Sí	Sí	No (el contador cambia)
12. Registrar vista	POST	No	No (cada llamada suma 1; conteo aproximado)	No

Leer una publicación (`GET`) no cuenta como vista: la vista es un `POST` aparte que envía el Sitio Web Público. Así el conteo sigue funcionando aunque la lectura salga de la caché. Registrar logs o aplicar el límite de tráfico son efectos secundarios aceptados de cualquier lectura.

Módulo Borradores (Drafts)

1. Creación de Borrador	
Ruta	**POST** `/api/v1/drafts`
Actor	Consola de Administración (`cms:write`)
Operación	`Idempotency-Key` (24 h) · Timeout 10 s · 60 peticiones/min
Reglas y errores	• `409` si `slug` ya existe en cualquier Post (borrador o publicado). • `422` si `title` no mide 5–120 caracteres, `content` está vacío o supera 100,000 caracteres, `slug` incumple el formato, o `tags` tiene más de 5 elementos, duplicados o formato inválido. • Si se omite `slug`, el sistema lo genera desde `title` (minúsculas, sin acentos, espacios a guiones); si el slug generado ya existe responde `409`. • Efecto: el sistema genera el `post_id` (UUIDv7); el Post nace con `status = BORRADOR` y `views_total = 0`.
Entra	`title` (!), `content` (!), `slug` (?), `tags[]` (?)
Sale	**201 Created:** `post_id`, `status` (BORRADOR), `title`, `slug`, `tags[]`, `created_at`, `updated_at`
Justificación	`Idempotency-Key` evita borradores duplicados si el cliente reintenta tras una falla de red. El slug se valida desde aquí para fallar temprano y no hasta el momento de publicar. El cliente nunca envía `post_id`: lo asigna el servidor.

2. Listado de Borradores	
Ruta	**GET** `/api/v1/drafts`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · Paginación por cursor · 60 peticiones/min
Reglas y errores	• Solo devuelve Posts con `status = BORRADOR`, ordenados por `updated_at` descendente y `post_id` como desempate. • `422` si `limit` está fuera de 1–50 o `cursor` es inválido. • Sin borradores: responde `200` con `items` vacío (no `404`).
Entra	`?cursor` (?), `?limit` (?, por defecto 20)
Sale	**200 OK:** `items[]` (`post_id`, `title`, `slug`, `tags[]`, `updated_at`), `next_cursor?`
Justificación	Corresponde al `getAll` del boceto que alimenta el panel del CMS. Se omite `content` para que el panel cargue rápido, y se pagina para que la respuesta no crezca sin límite. El cursor codifica `id` y `updated_at` para conservar el orden por fecha.

3. Obtención de un Borrador	
Ruta	**GET** `/api/v1/drafts/{post_id}`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si el Post ya está `PUBLICADO` (debe consultarse en `GET /posts/{post_id}`). • `422` si `post_id` no es un UUIDv7 válido.
Entra	`post_id` (en ruta)
Sale	**200 OK:** `post_id`, `title`, `slug`, `content`, `tags[]`, `status`, `created_at`, `updated_at`
Justificación	Devuelve el `content` completo para poder seguir editando, como describe el boceto. El `409` (y no `404`) indica claramente que el recurso existe pero cambió de módulo.

4. Actualización de Borrador	
Ruta	**PUT** `/api/v1/drafts/{post_id}`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · Idempotente por naturaleza (sin `Idempotency-Key`) · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si `status = PUBLICADO` (invariante de inmutabilidad editorial). • `409` si el `slug` enviado pertenece a otro Post. • `422` por las mismas validaciones de campos del endpoint 1 y si `post_id` no es un UUIDv7 válido. • PUT reemplaza `title` y `content`; si se omiten `slug` o `tags`, se conservan los actuales. Cada edición renueva `updated_at`.
Entra	`post_id` (en ruta), `title` (!), `content` (!), `slug` (?), `tags[]` (?)
Sale	**200 OK:** `post_id`, `status`, `title`, `slug`, `tags[]`, `updated_at`
Justificación	Como indica el boceto, este endpoint no modifica una publicación: actualiza el borrador mientras el autor sigue escribiendo. Por eso un Post publicado responde `409`.

5. Eliminación de Borrador	
Ruta	**DELETE** `/api/v1/drafts/{post_id}`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si `status = PUBLICADO` (las publicaciones se eliminan con `DELETE /posts/{post_id}`). • `422` si `post_id` no es un UUIDv7 válido. • Efecto: eliminación física del registro.
Entra	`post_id` (en ruta)
Sale	**204 No Content:** sin cuerpo de respuesta
Justificación	Un borrador no tiene lectores ni métricas, por lo que borrarlo no destruye información histórica y no requiere confirmación adicional.

6. Publicación de Borrador	
Ruta	**POST** `/api/v1/drafts/{post_id}/publish`
Actor	Consola de Administración (`cms:write`)
Operación	`Idempotency-Key` (24 h) · Timeout 10 s · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si el Post ya está `PUBLICADO` (con la misma `Idempotency-Key` dentro de 24 h devuelve el resultado original en lugar de `409`). • `422` si `content` tiene menos de 100 caracteres (invariante de completitud) o si `post_id` no es un UUIDv7 válido. • Efecto: cambia `status` a `PUBLICADO`, fija `published_at` y vuelve inmutables `title`, `slug`, `content` y `tags`. El Post pasa a ser gestionado por el módulo Posts.
Entra	`post_id` (en ruta); sin cuerpo
Sale	**200 OK:** `post_id`, `status` (PUBLICADO), `slug`, `url`, `published_at`
Justificación	Es el endpoint más importante: el único que cambia `status`, por lo que concentra las invariantes 1, 4 y 6. El identificador viaja en la ruta (no en el cuerpo, como `draftId` en el boceto), así que no hay cuerpo que validar. La respuesta devuelve `url` pero ya no un mensaje de texto: el código HTTP comunica el resultado.

Módulo Publicaciones (Posts)

7. Listado de Publicaciones	
Ruta	**GET** `/api/v1/posts`
Actor	Consola de Administración (`cms:write`), Sitio Web Público (`posts:read`)
Operación	Timeout 10 s · Paginación por cursor · 10 peticiones/s (Sitio Web Público)
Reglas y errores	• Solo devuelve Posts con `status = PUBLICADO`, ordenados por `published_at` descendente y `post_id` como desempate. • `422` si `limit`, `cursor` o `tag` tienen formato inválido. • Sin coincidencias: `200` con `items` vacío. • El listado no incluye `content`; para leer un artículo completo se usa el endpoint 8 (por `post_id`) o el 9 (por `slug`).
Entra	`?cursor` (?), `?limit` (?, por defecto 20), `?tag` (?), `If-None-Match` (?, cabecera)
Sale	• **200 OK:** `items[]` (`post_id`, `title`, `slug`, `tags[]`, `published_at`, `reading_time_min`), `next_cursor?` • Cabeceras: `Cache-Control: public, max-age=60`, `ETag`. • **304 Not Modified** si `If-None-Match` coincide con el `ETag` (sin cuerpo).
Justificación	Alimenta el panel de publicaciones activas del CMS y la página de artículos del blog. Al omitir `content`, la lista es ligera. El cursor codifica `id` y `published_at` para conservar el orden por fecha. La búsqueda por `slug` ya no se hace aquí (v1.1): se resuelve con el endpoint 9 en una sola petición.

8. Obtención de una Publicación por identificador	
Ruta	**GET** `/api/v1/posts/{post_id}`
Actor	Consola de Administración (`cms:write`), Sitio Web Público (`posts:read`)
Operación	Timeout 10 s · 10 peticiones/s (Sitio Web Público)
Reglas y errores	• `404` si `post_id` no existe **o** si el Post está en `BORRADOR` (invariante de visibilidad). • `422` si `post_id` no es un UUIDv7 válido.
Entra	`post_id` (en ruta)
Sale	• **200 OK:** `post_id`, `title`, `slug`, `content`, `tags[]`, `status` (PUBLICADO), `published_at`, `reading_time_min` • Cabeceras: `Cache-Control: public, max-age=60`, `ETag`. Acepta `If-None-Match`. • **304 Not Modified** si el `ETag` coincide (sin cuerpo).
Justificación	Responder `404` para borradores evita revelar a un lector que existe contenido sin publicar. No incluye `views_total`: ese dato pertenece al endpoint de métricas. La Consola lo usa porque conoce el `post_id` desde los listados.

9. Obtención de una Publicación por slug	
Ruta	**GET** `/api/v1/posts/slug/{slug}`
Actor	Sitio Web Público (`posts:read`), Consola de Administración (`cms:write`)
Operación	Timeout 10 s · 10 peticiones/s (Sitio Web Público)
Reglas y errores	• `404` si no existe un Post con ese `slug` **o** si el Post está en `BORRADOR` (invariante de visibilidad). • `422` si `slug` incumple la regex del diccionario de datos.
Entra	`slug` (en ruta)
Sale	• **200 OK:** `post_id`, `title`, `slug`, `content`, `tags[]`, `status` (PUBLICADO), `published_at`, `reading_time_min` • Cabeceras: `Cache-Control: public, max-age=60`, `ETag`. Acepta `If-None-Match`. • **304 Not Modified** si el `ETag` coincide (sin cuerpo).
Justificación	Añadido en la v1.1. El Sitio Web Público abre artículos desde la URL `/blog/{slug}`; antes necesitaba dos peticiones (buscar por `slug` en el listado y luego pedir el contenido por `post_id`). Ahora obtiene el artículo completo en una sola. La respuesta incluye `post_id` para que el sitio pueda registrar la vista (endpoint 12) sin consultas adicionales. Como el `slug` es único global (invariante 3), el resultado es siempre uno o ninguno. La ruta con el segmento fijo `slug` no colisiona con `/posts/{post_id}`.

10. Eliminación de Publicación	
Ruta	**DELETE** `/api/v1/posts/{post_id}`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si `status = BORRADOR` (los borradores se eliminan con `DELETE /drafts/{post_id}`). • `422` si falta `confirm=true` o si `post_id` no es un UUIDv7 válido. • Efecto: eliminación física del registro, incluyendo `views_total`; la URL pública deja de existir (`404`).
Entra	`post_id` (en ruta), `?confirm` (!, Boolean = `true`)
Sale	**204 No Content:** sin cuerpo de respuesta
Justificación	A diferencia de un borrador, aquí se pierden contenido y vistas, por lo que se exige `confirm=true` para evitar borrados accidentales. Si en el futuro se quiere conservar el histórico, bastaría añadir un estado `ARCHIVADO` en lugar de borrar.

11. Métricas de una Publicación	
Ruta	**GET** `/api/v1/posts/{post_id}/metrics`
Actor	Consola de Administración (`cms:write`)
Operación	Timeout 10 s · 60 peticiones/min
Reglas y errores	• `404` si `post_id` no existe. • `409` si `status = BORRADOR` (solo las publicaciones tienen métricas). • `422` si `post_id` no es un UUIDv7 válido. • Las métricas se calculan con datos propios del Post (no hay servicio analítico externo); `reading_time_min` es una estimación por número de palabras, no una medición del comportamiento del lector.
Entra	`post_id` (en ruta)
Sale	**200 OK:** `post_id`, `views_total`, `reading_time_min`, `published_at`
Justificación	Se separa de `GET /posts` porque, como anota el boceto, es “muy diferente que getAll”: es información numérica de un solo Post y no transporta contenido.

12. Registro de Vista	
Ruta	**POST** `/api/v1/posts/{post_id}/views`
Actor	Sitio Web Público (`posts:view`)
Operación	Timeout 10 s · 10 peticiones/s · Sin `Idempotency-Key`
Reglas y errores	• `404` si `post_id` no existe o el Post no está `PUBLICADO`. • `422` si `post_id` no es un UUIDv7 válido. • Efecto: incrementa `views_total` en 1 (invariante 5). • El conteo es aproximado: cuenta cargas del artículo, no visitantes únicos.
Entra	`post_id` (en ruta); sin cuerpo
Sale	**204 No Content:** sin cuerpo de respuesta
Justificación	Endpoint añadido: el campo `Vistas` del boceto necesita que alguien lo incremente, y solo el Sitio Web Público sabe cuándo se lee un artículo. Al ser un contador aproximado, un reintento duplicado es tolerable y no se exige `Idempotency-Key`; el límite de 10 peticiones/s acota el abuso. El sitio obtiene el `post_id` de la respuesta del endpoint 9.

PASO 6 DE 6: Tabla de Relevo y Priorización

Cada vecino identificado queda registrado con la obligación que esta API le impone, sin diseñar el vecino por dentro.

Vecino	Rol	Obligación impuesta (precondición)	Prio
Mecanismo de Autenticación	Infraestructura / Soporte	Emitir tokens con los scopes `cms:write`, `posts:read` y `posts:view`, y permitir que la API los valide en cada petición para responder `401` o `403` cuando corresponda.	2
Consola de Administración Web	Consumidor / Productor de contenido	Enviar `Idempotency-Key` en `POST /drafts` y `POST /drafts/{post_id}/publish`; respetar los bloqueos de estado (no editar publicaciones); enviar `confirm=true` al eliminar publicaciones; tratar `post_id` y `cursor` como cadenas opacas y paginar con el `next_cursor` recibido.	3
Sitio Web Público DevBlog_	Consumidor (lectura y registro de vistas)	Usar un token de solo lectura; consultar únicamente `/posts` y `/posts/slug/{slug}`; abrir cada artículo con una sola petición por `slug`; enviar una sola vista por carga de artículo usando el `post_id` recibido; manejar `404` cuando un artículo fue eliminado y respetar el límite de 10 peticiones/s.	3

Justificación de diseño

Sin prioridad 1: no existe un servicio automático que origine la verdad y dispare escrituras hacia esta API; el único productor de estado es el autor a través de la Consola.

Orden: primero la infraestructura (sin autenticación definida no puede operar ningún cliente) y después las capas de presentación. Entre ambas, la Consola va antes que el Sitio Web Público porque produce el contenido que el sitio muestra.

ANEXO A: Trazabilidad del boceto al contrato

Resumen de cómo cada elemento del boceto original (libreta del 15/10/26) se convirtió en el contrato final y por qué.

Boceto original	Contrato final	Razón del cambio
`POST /Drafts/Create`	`POST /api/v1/drafts`	Ruta con sustantivo y verbo HTTP, versionada; se agregan validaciones e idempotencia.
`PUT /Drafts/modify`	`PUT /api/v1/drafts/{post_id}`	El identificador va en la ruta; se bloquea la edición de publicaciones.
`GET /Drafts/get?id=`	`GET /api/v1/drafts/{post_id}`	Mismo propósito; id como parámetro de ruta.
`GET /Drafts/getAll`	`GET /api/v1/drafts`	Se añade paginación por cursor con `id` y `updated_at`.
`DELETE /Drafts/delete`	`DELETE /api/v1/drafts/{post_id}`	Se define el error `409` para publicaciones.

Boceto original	Contrato final	Razón del cambio
`POST /Drafts/publish` con `{ "draftId": 105 }`	`POST /api/v1/drafts/{post_id}/publish`	Id en ruta; respuesta con `status`, `url` y `published_at` en lugar de un mensaje.
`GET /Posts/get?id=`	`GET /api/v1/posts/{post_id}` y `GET /api/v1/posts/slug/{slug}`	Se oculta cualquier Post en `BORRADOR`; se añade la lectura por slug para que el sitio público lea con una sola petición.
`DELETE /Posts/delete`	`DELETE /api/v1/posts/{post_id}`	Se exige `confirm=true` por ser un borrado irreversible.
`GET /Posts/getAll`	`GET /api/v1/posts`	Paginación por cursor con `id` y `published_at`, y filtro `tag`.
`GET /Posts/metrics`	`GET /api/v1/posts/{post_id}/metrics`	Se precisa que es por Post y solo para publicados.
`POST /Posts/ads`	Eliminado	Sin consumidor: el DevBlog no inserta anuncios. No hay atributos ni endpoints de anuncios en el contrato.
(no existía)	`POST /api/v1/posts/{post_id}/views`	Alguien debe incrementar `Vistas`.
Tabla `Publicacion_Blog` con `Ads_Activos`	Mismo modelo de una sola tabla, sin `Ads_Activos`; con `updated_at` y `published_at`	Coherente con la exclusión de anuncios; fechas necesarias para orden y métricas.
`ID` Entero (PK)	`post_id` UUIDv7	Evita que se pueda enumerar la base de datos y deducir el total de artículos.
`Etiquetas` (Texto)	`tags` (lista de 0 a 5 strings validados)	Contrato más estricto; el almacenamiento interno queda libre.

ANEXO B: Registro de cambios de la versión 1.0 a la 1.1

#	Corrección	Dónde se aplicó
1	**Lectura en una sola petición.** Nuevo endpoint `GET /api/v1/posts/slug/{slug}` que devuelve el artículo completo. Se retiró el filtro `?slug` del listado y los endpoints posteriores se renumeraron (eliminación 10, métricas 11, vistas 12).	Paso 1 (operaciones del Sitio Web Público), Paso 5 (endpoints 7 y 9 a 12), Paso 6, Anexo A.
2	**Privacidad del identificador.** `post_id` pasa de entero autoincremental a UUIDv7, para que ningún visitante pueda enumerar los endpoints y deducir cuántos artículos existen. Nueva invariante 9.	Paso 3 (atributos, invariante 9 y justificación), Paso 4 (diccionario), Paso 5 (`422` por UUIDv7 inválido), Anexo A.
3	**Cursor de paginación.** El cursor opaco codifica el `id` y el timestamp de ordenamiento (`updated_at` o `published_at`); `post_id` actúa como desempate. Se muestra un ejemplo real en base64url y su estructura decodificada.	Paso 4 (diccionario y sección “Estructura del cursor”), Paso 5 (reglas transversales y endpoints 2 y 7).
4	**Coherencia de alcance.** Sin cambios: se mantiene la exclusión total de anuncios; no hay atributos ni endpoints de ads en el modelo ni en la API.	Paso 1 (límites y justificación), Anexo A.

ANEXO C: Registro de cambios de la versión 1.1 a la 1.2

#	Cambio	Dónde se aplicó
1	**Estilo arquitectónico documentado:** por qué REST y no GraphQL, nivel 2 de Richardson y compromisos de REST asumidos (vocabulario limitado, dos colecciones del mismo recurso, ausencia de estado y versionado).	Paso 1 (sección “Estilo arquitectónico: REST”).
2	**Estrategia de caché HTTP:** `Cache-Control` y `ETag` en las lecturas públicas, `304 Not Modified` con `If-None-Match` y `no-store` en todo lo demás.	Paso 5 (reglas transversales y endpoints 7, 8 y 9).
3	**Propiedades HTTP por endpoint:** seguro, idempotente y cacheable, con la aclaración de por qué leer no cuenta una vista.	Paso 5 (tabla antes de los módulos).
4	**Tipo de medio y versionado explícitos:** `application/json` y política de cuándo se crearía `/api/v2`.	Paso 5 (reglas transversales).
5	**Sin cambios** en el modelo de datos, las rutas, los errores ni las invariantes: la v1.2 es compatible con la v1.1.	—

ANEXO D: Implementación de referencia y conexión con el DevBlog

Este anexo no modifica el contrato de la v1.2. Su objetivo es documentar cómo se implementó el contrato, cómo se conectó el Sitio Web Público y qué decisiones operativas se tomaron para que la API pueda ejecutarse, verificarse y reutilizarse. Los endpoints, el modelo de datos, las invariantes, los códigos de error y las reglas transversales del Paso 5 siguen siendo exactamente los de la v1.2.

D.1 Qué se conserva del contrato

La implementación respeta el contrato; no introduce rutas de contenido nuevas ni cambia las existentes.

Elemento del contrato	Estado en la implementación
Rutas `/api/v1/drafts` y `/api/v1/posts`	Sin cambios. Se implementan los 12 endpoints del Paso 5.
Modelo Post y estados	Sin cambios. Un solo Post transita de BORRADOR a PUBLICADO.
Invariantes 1–9	Sin cambios. Se aplican en `app/services.py` y se prueban automáticamente.
Formato de error	Sin cambios: `{"error": {"code", "message", "field?"}}`.
Paginación por cursor	Sin cambios en el contrato. El cursor ahora se firma para detectar alteraciones.
ETag, Cache-Control y 304	Sin cambios. Se mantienen en los endpoints 7, 8 y 9.
Idempotencia	Sin cambios en la semántica. Ahora usa un lock por clave, no un lock global.
Autenticación y scopes	Sin cambios: Bearer `cms:write`, `posts:read`, `posts:view`.
Límites de tráfico	Sin cambios: 60/min Consola y 10/s Sitio Web Público.
Versionado	Sin cambios: prefijo `/api/v1`.

D.2 Arquitectura implementada

La separación por capas permite cambiar la persistencia o la configuración sin tocar las reglas de negocio.

Capa	Archivo(s)	Responsabilidad
Configuración	`app/config.py`, `.env.example`	Identidad, tokens, scopes, límites, base de datos, CORS y URL pública.
Entrada HTTP	`app/main.py`, `app/routers/*.py`	Creación de la app, CORS, routers, `/health` y dependencias.
Contrato y validación	`app/schemas.py`	Modelos Pydantic de entrada/salida, `extra=forbid` y validaciones.
Dominio	`app/domain.py`	Entidad Post, estados, UUIDv7, slug, fechas y tiempo de lectura.
Negocio	`app/services.py`	Invariantes, transiciones y reglas de publicación/visibilidad.
Persistencia	`app/repository.py`, `app/sqlite_repository.py`	Interfaz `PostRepository` e implementación SQLite.

Capa	Archivo(s)	Responsabilidad
Infraestructura	`app/infra.py`, `app/caching.py`, `app/pagination.py`, `app/errors.py`	Rate limit, idempotencia, caché HTTP, cursor firmado y errores.
Datos iniciales	`app/seed.py`	Carga idempotente de tres artículos de ejemplo.
Pruebas	`tests/`	43 pruebas del contrato, persistencia, cursor, CORS, salud y concurrencia.

D.3 Persistencia SQLite

La primera versión guardaba en memoria y perdía el contenido al reiniciar. La entrega usa **SQLite**: un archivo local, sin servidor de base de datos, suficiente para un blog de un autor. La interfaz `PostRepository` se mantiene, así que el servicio no sabe si los datos están en memoria, SQLite o PostgreSQL.

Columna	Tipo SQLite	Regla / uso
`post_id`	TEXT PRIMARY KEY	UUIDv7 generado por el servidor; nunca lo envía el cliente.
`title`	TEXT NOT NULL	5–120 caracteres, sin saltos de línea.
`slug`	TEXT NOT NULL UNIQUE	kebab-case, 3–100 caracteres, único global.
`content`	TEXT NOT NULL	Markdown, 1–100,000 caracteres; mínimo 100 para publicar.
`tags`	TEXT NOT NULL	Array JSON de 0–5 etiquetas únicas.
`status`	TEXT NOT NULL CHECK	Solo `BORRADOR` o `PUBLICADO`.
`created_at`	TEXT NOT NULL	ISO 8601 UTC; no cambia.
`updated_at`	TEXT NOT NULL	ISO 8601 UTC; se renueva al editar un borrador.
`published_at`	TEXT NULL	ISO 8601 UTC; se fija una sola vez al publicar.
`views_total`	INTEGER NOT NULL DEFAULT 0	Entero ≥ 0; +1 por cada vista.

Se crean tres índices: `UNIQUE(slug)`, `(status, updated_at DESC, post_id DESC)` y `(status, published_at DESC, post_id DESC)`. Se activa WAL, `busy_timeout` y un `RLock` para serializar escrituras. El archivo por defecto es `data/devblog.db`.

Para cambiar a PostgreSQL/MySQL se implementa otra clase que herede de `PostRepository` y se pasa en `create_app(repository=...)`; ni el servicio ni los routers cambian.

D.4 Configuración y reutilización

El código no está atado al DevBlog. La identidad, los tokens, la base, CORS y la URL pública se leen de variables de entorno o de un archivo `.env`. Los nombres antiguos `DEVBLOG_*` siguen funcionando como alias para no romper instalaciones existentes.

Variable	Valor por defecto	Propósito
`CMS_APP_NAME`	`DevBlog CMS API`	Nombre que aparece en `/docs`.

Variable	Valor por defecto	Propósito
`CMS_ENV`	`development`	Entorno; `production` activa avisos de seguridad.
`CMS_DB_PATH`	`data/devblog.db`	Archivo SQLite. `:memory:` solo para pruebas.
`CMS_CONSOLE_TOKEN`	`console-dev-token`	Token de escritura de la Consola.
`CMS_PUBLIC_TOKEN`	`public-dev-token`	Token de lectura y vistas del Sitio Web Público.
`CMS_CURSOR_SECRET`	valor de desarrollo	Secreto HMAC de los cursores de paginación.
`CMS_CORS_ORIGINS`	orígenes locales	Orígenes permitidos, separados por coma.
`CMS_PUBLIC_SITE_URL`	vacío	Dominio público para construir `url` absoluta.
`CMS_PUBLIC_ARTICLE_PATH`	`/blog/{slug}`	Ruta del artículo; debe contener `{slug}`.
`CMS_IDEMPOTENCY_TTL_S`	`86400`	Ventana de idempotencia (24 h).
`CMS_CONSOLE_RATE_LIMIT`	`60`	Peticiones por ventana de la Consola.
`CMS_PUBLIC_RATE_LIMIT`	`10`	Peticiones por ventana del Sitio Web Público.
`CMS_LOG_LEVEL`	`INFO`	Nivel de logs.

Para reutilizar la API en otro proyecto: copiar la carpeta, crear un `.env` nuevo y cambiar `CMS_APP_NAME`, `CMS_DB_PATH`, `CMS_CONSOLE_TOKEN`, `CMS_PUBLIC_TOKEN`, `CMS_CURSOR_SECRET`, `CMS_CORS_ORIGINS` y `CMS_PUBLIC_ARTICLE_PATH`. No se añadió un campo `site_id` porque el contrato v1.2 es de un solo blog; un multi-sitio exigiría cambiar el modelo y publicar una nueva versión.

D.5 Decisiones operativas que complementan el contrato

Tema	Decisión implementada	Motivo	Efecto en el contrato
Persistencia	SQLite con la misma interfaz `PostRepository`.	Evitar perder contenido al reiniciar.	Ninguno: mismos endpoints y mismos datos.
CORS	Orígenes configurables con `CMS_CORS_ORIGINS`.	El blog es un origen distinto de la API.	Ninguno: solo agrega cabeceras CORS.
Cursor	HMAC-SHA256 sobre el payload base64url.	Cumplir la regla “cursor alterado → 422”.	Ninguno: la parte decodificable sigue siendo `id` + timestamp.
Idempotencia	Lock por clave y scope por consumidor.	Evitar que una operación lenta bloquee todas las demás.	Ninguno: misma semántica de replay y 422.
Error 500	Handler genérico con `INTERNAL_ERROR` y `no-store`.	Mantener el formato de error también en fallos inesperados.	Ninguno: nuevo código operativo, no de negocio.
Rate limit	Dependencia de autenticación compartida.	Evitar contar dos veces la misma petición.	Ninguno: mismos límites 60/min y 10/s.
URL pública	`CMS_PUBLIC_ARTICLE_PATH` y `CMS_PUBLIC_SITE_URL`.	Conectar el blog estático sin fijar `/blog/{slug}`.	`url` sigue siendo un enlace relativo o absoluto; el valor por defecto es `/blog/{slug}`.

Tema	Decisión implementada	Motivo	Efecto en el contrato
Salud	`GET /health` fuera de `/api/v1`.	Diagnóstico rápido de API y base de datos.	Endpoint operativo, no forma parte del contenido.
Seed	Tres artículos Markdown de ejemplo.	Tener contenido para la demostración.	Ninguno: son Posts normales.

D.6 Conexión con el Sitio Web Público

El blog sigue siendo estático: la API no renderiza HTML. La integración se hace en el navegador con `fetch` y el token público. La portada pide el listado, el artículo se abre por slug y la vista se registra con un `POST` separado, tal como exige el contrato.

Paso	Acción del sitio	Petición	Respuesta usada
1	Cargar la portada	`GET /api/v1/posts?limit=6`	`items[]` con `post_id`, `title`, `slug`, `tags`, `published_at`, `reading_time_min`.
2	Abrir un artículo	`GET /api/v1/posts/slug/{slug}`	Post completo, incluido `content`.
3	Renderizar Markdown	—	HTML seguro generado en el navegador.
4	Registrar la vista	`POST /api/v1/posts/{post_id}/views`	204 No Content.
5	Si la API falla	—	Se conservan las tarjetas estáticas de la portada.

Archivo del blog	Función
`dev-blog/api-config.js`	URL base de la API y token público.
`dev-blog/api.js`	Cliente `fetch`, manejo de errores, idempotencia y renderizador Markdown seguro.
`dev-blog/notes.js`	Carga las publicaciones en la portada y aplica el fallback estático.
`dev-blog/articulo.html` + `articulo.js`	Artículo dinámico por `?slug=`, etiquetas, fecha, tiempo de lectura y registro de vista.
`dev-blog/index.html`	Integra la sección de notas con la API y enlaza la Consola CMS.
`dev-blog/article.css`	Estilos del artículo dinámico y del contenido Markdown.

El token público está en `api-config.js` porque el navegador lo necesita; es de solo lectura y está limitado a 10 peticiones/s. El token de la Consola **no** se escribe en el código: se introduce en `admin.html` y se guarda solo en `sessionStorage` mientras dura la pestaña.

La portada usa `cache: "no-store"` para mostrar publicaciones recién creadas. La API sigue enviando `ETag` y `Cache-Control` y el artículo conserva la caché de 60 s del contrato; por eso, tras eliminar una publicación, su URL puede seguir sirviéndose desde la caché del navegador hasta 60 s.

D.7 Consola de Administración Web

La consola es una página estática (`admin.html`) que usa la misma API con el token `cms:write`. No hay un backend adicional ni una segunda lógica de negocio.

Capacidad de la consola	Endpoint	Token
Validar el token al conectar	`GET /drafts?limit=1`	`cms:write`
Listar borradores	`GET /drafts`	`cms:write`
Crear borrador	`POST /drafts` + `Idempotency-Key`	`cms:write`
Editar borrador	`PUT /drafts/{post_id}`	`cms:write`
Publicar borrador	`POST /drafts/{post_id}/publish` + `Idempotency-Key`	`cms:write`
Eliminar borrador	`DELETE /drafts/{post_id}`	`cms:write`
Listar publicaciones	`GET /posts`	`cms:write`
Ver detalle de una publicación	`GET /posts/{post_id}`	`cms:write`
Consultar métricas	`GET /posts/{post_id}/metrics`	`cms:write`
Eliminar publicación	`DELETE /posts/{post_id}?confirm=true`	`cms:write`

Flujo de uso: abrir `admin.html`, introducir el token, crear un borrador, guardarlo, publicarlo, comprobarlo en el blog, revisar sus métricas y, si hace falta, eliminarlo. La consola usa `cache: "no-store"` en sus lecturas para ver el estado actual después de publicar.

D.8 Pruebas de aceptación

Prueba	Qué verifica	Resultado
Suite automática	Contrato, invariantes, validaciones, auth, rate limit, caché y paginación.	43/43 pruebas.
Persistencia	Crear y publicar en una instancia; leer en otra instancia con el mismo archivo.	OK.
Cursor alterado	Cambiar el payload o la firma del cursor.	422 `INVALID_CURSOR`.
CORS	Preflight desde el origen del blog.	Cabecera `Access-Control-Allow-Origin` correcta.
Error 500	Fallo simulado en el servicio.	`INTERNAL_ERROR` + `Cache-Control: no-store`.
Idempotencia concurrente	Ocho hilos con la misma clave y la misma petición.	La operación se ejecuta una sola vez.
Claves distintas	Dos operaciones con claves distintas en paralelo.	No se bloquean entre sí.
Integración manual	Portada, artículo, vista, consola, crear, editar, publicar y eliminar.	OK.

Al terminar las pruebas manuales se eliminaron los datos temporales. La base quedó con los tres artículos de ejemplo y cero borradores.

D.9 Cómo ejecutar y verificar

API (Windows PowerShell):

```
cd API/devblog-cms-api-1.2/devblog-cms-api
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python -m app.seed
uvicorn app.main:app --reload --port 8080
```

Servicio de métricas (otra terminal):

```
cd pendiente/metrics-navigation-main/metrics-navigation-main
.venv\Scripts\activate
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Generador estático (otra terminal):

```
cd pendiente/static-sites-generator-main/static-sites-generator-main
.venv\Scripts\activate
uvicorn src.main:app --host 127.0.0.1 --port 8001
```

Sitio web (otra terminal):

```
cd dev-blog
python -m http.server 5500
```

Atajo: desde la raíz del proyecto, `.\start-all.ps1` abre los cuatro servicios en ventanas separadas de PowerShell. Para detenerlos, cierra cada ventana o pulsa `Ctrl+C`.

Recurso	URL
CMS API	http://127.0.0.1:8080
Documentación del CMS	http://127.0.0.1:8080/docs
Salud del CMS	http://127.0.0.1:8080/health
Métricas	http://127.0.0.1:8000
GraphiQL de métricas	http://127.0.0.1:8000/v1/query
Generador estático	http://127.0.0.1:8001
Swagger del generador	http://127.0.0.1:8001/docs
Blog	http://127.0.0.1:5500/
Consola CMS	http://127.0.0.1:5500/admin.html

Tokens: `console-dev-token` (Consola, `cms:write`, 60/min) y `public-dev-token` (Sitio Web Público, `posts:read` y `posts:view`, 10/s).

La base está en `data/devblog.db`. Para reiniciar el contenido: detener la API, borrar el archivo y ejecutar `python -m app.seed`. En producción se deben cambiar los tokens y el secreto del cursor. El CMS usa el puerto 8080 para coexistir con Métricas (8000) y el Generador (8001).

D.10 Límites y trabajo futuro

Tema	Estado	Nota
GraphQL del CMS	No implementado en el CMS	El contrato v1.2 sigue en REST. El servicio de métricas sí expone GraphQL en `/v1/query`.
Métricas avanzadas	En el servicio de métricas	El CMS conserva `views_total` y `reading_time_min`; el colector aporta pageviews, sesiones, rutas, referrers y engagement.
Servicios del ecosistema	Desplegados	Métricas en 8000 y Generador en 8001; CMS en 8080 y blog en 5500.
Timeout de 10 s / 504	Delegado al gateway	La app no simula un 504; lo aplica el servidor/proxy.
Rate limit e idempotencia distribuidos	En memoria por proceso	Con varias instancias deben pasar a Redis u otro almacén compartido.
Multi-sitio	No implementado	Requeriría `site_id` y una nueva versión del contrato.
Autenticación real	Tokens de desarrollo	El contrato prevé un servicio emisor externo de tokens.
Caché pública	60 s	Una publicación eliminada puede servirse desde caché hasta 60 s.
Consola	Token manual	En producción debe ir detrás de un login real.

D.11 Checklist de entrega

Requisito	Evidencia
12 endpoints del contrato	`app/routers/drafts.py`, `app/routers/posts.py` y `/docs`.
Modelo e invariantes	`app/domain.py`, `app/services.py` y `tests/test_api.py`.
Persistencia	`app/sqlite_repository.py`, `data/devblog.db`.
Autenticación y scopes	`app/dependencies.py`.
Rate limit	`app/infra.py` y pruebas de 429.
Idempotencia	`app/infra.py`, `tests/test_infra.py`.
Caché HTTP	`app/caching.py` y pruebas de ETag/304.
Paginación	`app/pagination.py` y pruebas de cursor.
Formato de error	`app/errors.py`.
Configuración reutilizable	`app/config.py` y `.env.example`.
Datos iniciales	`app/seed.py`.

Requisito	Evidencia
Sitio Web Público	`dev-blog/api.js`, `notes.js`, `articulo.html`, `articulo.js`.
Consola de Administración	`dev-blog/admin.html`, `admin.js`, `admin.css`.
Servicio de métricas desplegado	`pendiente/metrics-navigation-main`; `/health` y GraphiQL en 8000.
Generador estático desplegado	`pendiente/static-sites-generator-main`; `/health` y `/docs` en 8001.
Beacon de métricas en el blog	`dev-blog/metrics.js` y `dev-blog/api-config.js`.
Generación estática desde la consola	`admin.html`, `admin.js` y `scripts/generate_static.py`.
Pruebas	`tests/`; 43 pruebas del CMS, 13 de métricas y 4 del generador.
Documentación	`README.md`, `pendiente/README_DESPLIEGUE.md` y este Anexo D.

D.12 Servicios del ecosistema: métricas y generador estático

Además del CMS, la entrega despliega los dos servicios del ecosistema del profesor. No se reescribieron: se ejecutan desde sus repositorios y se conectaron con el blog y la consola. El CMS ocupa el puerto 8080 para dejar libres el 8000 (métricas) y el 8001 (generador).

Servicio	Repositorio	Puerto	Rol	Estado
Metrics Collector & Retriever	`pendiente/metrics-navigation-main`	8000	Beacon, tráfico, agentes y GraphQL.	Desplegado y probado.
Static Sites Generator	`pendiente/static-sites-generator-main`	8001	Compila snapshots en HTML con SHA-256.	Desplegado y probado.

Integración	Cómo se conecta	Evidencia
Beacon del blog	`dev-blog/metrics.js` inyecta `/v1/beacon.js` con `data-key` y `data-site="devblog"`.	Pageviews en GraphQL para `/` y `/articulo.html`.
Consulta de tráfico	GraphQL `/v1/query` con `trafico(sitioSlug: "devblog", ...)`.	`pageviews`, `sesiones`, `visitantesUnicosEstimados` y `porRuta`.
Generación estática	Botón de la consola o `scripts/generate_static.py` → `POST /api/v1/pages`.	`COMPLETED`, motor `astro`, SHA-256 y HTML en `/content`.
Artefactos locales	`dev-blog/generated/{slug}.html`.	Tres archivos generados para los artículos de ejemplo.

Consulta GraphQL de ejemplo para verificar el beacon:

```
{
  trafico(sitioSlug: "devblog", desde: "2026-11-04T00:00:00Z",
          hasta: "2026-11-06T00:00:00Z", grano: DIA) {
    pageviews
    sesiones
    visitantesUnicosEstimados
    porRuta { clave valor }
  }
}
```

Prueba de integración	Resultado
Suite del servicio de métricas	13/13 pruebas.
Suite del generador estático	4/4 pruebas.
Beacon en portada y artículo	2 pageviews registrados en GraphQL.
Generación estática desde la consola	3 artefactos con `COMPLETED` y SHA-256.
Suite del CMS	43/43 pruebas.

Corrección local aplicada	Detalle
`detectar_entorno` en métricas	Reconoce loopback con puerto (`127.0.0.1:5500`), no solo `localhost:5500`.
CORS de métricas	`allow_credentials=True` para que `navigator.sendBeacon` no sea bloqueado.

El colector corre con `STORAGE=memory`: los eventos se pierden al reiniciar. Para persistencia real se configura `STORAGE=mongo` y `MONGO_URI`. El generador no tiene autenticación y su CORS es abierto; está pensado para local y en producción debe quedar detrás de un proxy/autenticación.

Justificación de diseño

El contrato no cambió: se implementó. Las decisiones de este anexo son operativas y están aisladas del contrato. SQLite se eligió porque el volumen es bajo y no requiere servidor; CORS porque el blog es un origen distinto; el cursor firmado porque la especificación exige 422 ante un cursor alterado; la idempotencia por clave porque el lock global castigaba peticiones independientes.

La configuración por entorno permite reutilizar la API en otro proyecto sin tocar el dominio. La Consola y el artículo dinámico son consumidores de la misma API, no una segunda lógica de negocio.

Los servicios del ecosistema se desplegaron sin reescribirlos: Métricas en 8000 aporta beacon, tráfico y GraphQL; el Generador en 8001 produce artefactos HTML con SHA-256. El CMS solo los consume.

Alcance: el contrato v1.2 del CMS se mantiene como fuente de verdad. Las métricas avanzadas y GraphQL viven en el servicio de métricas, no en el CMS.